

Supplementary Material

Reproducibility guide for an $8 + 9$ gluing computation
for the Ramsey number $R(F_3, F_4)$

Hanxin Jiang Zehui Shao

Institute of Computing Science and Technology,
Guangzhou University, Guangzhou 510006, China

1 Purpose and verification scope

This supplement explains how to compile the search program, reproduce the seed catalogues and filtering counts, run the distributed search, and verify the certificates for the final forty seed pairs. Here F_t is the graph formed by t triangles with a common vertex and otherwise disjoint vertex sets. A colouring is admissible if it contains neither a red F_3 nor a blue F_4 . The mathematical justification of the reductions is given in the accompanying article.

The bulk-search results and cumulative solver counts reported in the article were obtained by running the C++ search program on the authors' computing cluster. The local certificate checks described below were performed separately after the distributed search. Thus the cluster statistics and the subsequent independent verification results have distinct computational provenance.

The archived search has 88 294 674 retained ordered pairs of seeds on eight and nine vertices. The reported bulk computation excludes 88 294 634 of these pairs. The remaining forty are excluded separately: thirty-four by the counting argument in the article, with its finite premises checked from the seed graphs, and six by independently replayable finite-domain certificates. The quick verification below establishes these forty local exclusions. It does not rerun or certify the other 88 294 634 solver decisions or prove catalogue completeness by itself.

The distribution provides the inputs and source needed for a fresh full computation. It does not include the complete historical cluster checkpoints or a checked propositional proof for each bulk exclusion. The supplied source, the completeness arguments in the article, and a full rerun therefore have different roles from the local certificate checker. File checksums establish file identity, not mathematical truth.

2 Files and conventions

Extract `fan_F3F4_reproducibility.zip` to a writable directory. All commands below are issued from the extracted directory containing `README.txt`, unless a different working directory is stated. The shell examples use Linux syntax. Section 5 gives the corresponding Windows build procedure.

Location	Contents and purpose
<code>src/</code>	C++17 search source, build files, and required third-party source and licences.
<code>data/</code>	Fixed graph6 seed files, the ordered-pair catalogue, and the dataset identity.
<code>tools/</code>	Canonical generation, catalogue preparation, and independent data-validation programs.
<code>certificates/</code>	The forty case specifications and final counting and finite-domain certificate data.
<code>verify40.py</code>	Entry point for independent verification of the forty exceptional pairs.
<code>combine_results.py</code>	Checks coverage of a fresh full run and combines it with the verified local exclusions.
<code>docs/</code>	This supplement in editable LaTeX and PDF form.

A seed index and a case index are zero-based. Machine numbers are one-based. The red graph determines the colouring; all other edges of the complete graph are blue. A graph6 record encodes an undirected graph, using the vertex order fixed by the corresponding seed file. The production dataset identifier is `e38c0723b6a3be74`; the engine identifier is `fan17-cpp-1`. The eight-vertex and nine-vertex sides are ordered: there is no transposition identification between them.

The published files should be kept unchanged. Put new checkpoints, timing logs, generated certificates, and validation reports in separate working directories. These outputs are produced by the reader's run and are not prerequisites for the archived quick check.

3 Requirements

The forty-case checker requires Python 3.9 or later and only its standard library. The C++ search requires a C++17 compiler and threads. GCC 9 or later, a recent Clang, or the x64 Microsoft Visual C++ toolchain can be used. A CMake build requires CMake 3.15 or later. The included direct build scripts provide an alternative to CMake. MiniSat and the JSON header are supplied in the archive; no separate SAT-solver installation is needed.

Full seed and profile regeneration requires Python 3.11 or later and the pinned Python packages in `tools/requirements.txt`. Graph isomorphism is handled by the BLISS backend exposed through `igraph`. Verification of the six finite-domain certificates uses neither BLISS nor a SAT solver. No GPU is required. Select the thread count to match the CPU resources allocated by the cluster scheduler.

A full run writes 176 590 checkpoint files. Use storage with adequate space and support for atomic file replacement. The total runtime depends on the hardware, thread count, time limits and difficult instances; a short pilot is not a reliable prediction of completion time. No historical whole-cluster runtime is asserted in this supplement.

4 Quick verification of the forty exceptional pairs

Run:

```
python3 verify40.py
```

On Windows, use `python` instead of `python3` when that is the installed command. The program checks the fixed input identities, the binding of case indices to the seed catalogue, the seed adjacency data, the finite premises for the thirty-four counting exclusions, and all six certificate trees. The required outcome is a successful process exit and a report containing forty excluded cases, with zero unresolved cases in this local set. The default report is `results/verified40.json`; an alternative path may be selected by `-report`.

For the six certificates the expected aggregate replay counts are 7 499 search nodes, 198 288 explicit fan witnesses, and 448 770 domain deletions. Each fan witness specifies a centre and three or four disjoint pairs of peripheral vertices. The checker tests the colours of all spokes and matching edges directly. It also checks every domain deletion and every branch of the contradiction tree. An unknown edge in a partial colouring belongs to neither colour.

The thirty-four counting exclusions and six tree certificates partition the same fixed set of forty cases. A passed local check must not be interpreted as a passed full-catalogue check. The full-run combination procedure in Section 9 checks this coverage separately.

5 Compiling and checking the search program

5.1 CMake build

From the archive root, run:

```
cmake -S src -B build -DCMAKE_BUILD_TYPE=Release
cmake --build build --config Release --parallel 4
```

With a single-configuration Linux generator, the executable is `build/fan_ramsey17`. Test it and inspect the fixed catalogue:

```
./build/fan_ramsey17 self-test
./build/fan_ramsey17 catalog --data data
```

With the Visual Studio generator on Windows, use `build\Release\fan_ramsey17.exe` in place of `./build/fan_ramsey17`. Run the Windows commands in a developer terminal with the x64 C++ toolchain available.

5.2 Direct builds

On Linux, the alternative command

```
bash src/build_linux.sh
```

creates `src/fan_ramsey17`. In the remaining examples replace `./build/fan_ramsey17` by `./src/fan_ramsey17` when using this build.

In the x64 Native Tools Command Prompt for Visual Studio, the equivalent Windows procedure is:

```
pushd src
call build_windows.bat
popd
src\fan_ramsey17.exe self-test
src\fan_ramsey17.exe catalog --data data
```

Here the executable for subsequent commands is `src\fan_ramsey17.exe`. Use ordinary single-line commands in Windows; a backslash used for Linux line continuation is not a Windows line-continuation character.

5.3 Expected catalogue dimensions

Quantity	Expected value
Eight-vertex seeds	8 812
Nine-vertex seeds	115 174
Eight-vertex profiles	6 860
Nine-vertex profiles	60 041
Retained ordered seed pairs	88 294 674
Cases per full task	500
Number of tasks	176 590
Cases in the last task	174

The executable rejects a catalogue or seed file that differs from its fixed dataset. It also checks catalogue dimensions and basic profile fields. These checks are useful consistency tests, not an independent proof of exhaustive generation.

For a small executable test that does not start the full search, run:

```
./build/fan_ramsey17 run --data data \
--machine 1 --machines 1 --threads 1 \
--task-ids 0 --case-budget 5 --seconds 5 \
--symmetry on --results results/smoke
```

This deliberately processes at most five cases. Use a different results directory for a new independent full computation.

6 Running a full computation on one machine

```
./build/fan_ramsey17 run --data data \
--machine 1 --machines 1 --threads 4 \
--symmetry on --seconds 5 --results results/machine01
```

The `-seconds` option is a time limit for each case, not for the whole job. The `-threads` option controls concurrent workers on this machine. The symmetry option activates the symmetry constraints described in the article. An independent run with `-symmetry off` is supported but may be slower. The default task order, `spread`, is a deterministic permutation of all assigned tasks and is not a sampling rule.

The status fields have the following meanings:

Field	Interpretation
PENDING	Cases not yet processed.
UNSAT	Cases for which the SAT solver reports that no permitted completion exists.
UNKNOWN	Cases for which the time-limited attempt did not decide satisfiability. These are not exclusions.
SAT	Cases with a candidate completion; the program checks and stores its red adjacency data.

Each task progress line reports counts within that task. Its `wall_seconds` is elapsed time since the current run began; it is not the duration of that task alone. The final run line reports the number of attempts in the invocation. The runner may finish normally with unresolved cases because every assigned case has received its time-limited attempt.

The flag `proof_verified=false` is intentional in production solver output: the runner does not emit and independently check a propositional proof for every exclusion. A solver's negative result, a repeated negative result and a replayed certificate are different forms of evidence.

7 Resuming, retrying and merging

Repeating a `run` command with the same results directory resumes pending work. Already resolved cases are skipped. Previously unresolved cases are attempted again only if `-retry-unknown` is present. For example, after the first pass has stopped:

```
./build/fan_ramsey17 run --data data \  
--machine 1 --machines 1 --threads 4 \  
--symmetry on --seconds 60 --retry-unknown \  
--results results/machine01
```

Further passes may use 600 or 3600 seconds per case. These are example retry settings, not a guarantee that every case will finish. Keep the dataset and seed ordering unchanged between passes.

After a run has stopped, create a summary and a merged checkpoint set:

```
./build/fan_ramsey17 merge --data data \  
--inputs results/machine01 --output results/merged_01
```

The output directory must be new or contain no checkpoint files. The merge writes `summary.json`, `pending_tasks.json`, `unknown_tasks.json`, and the merged task files. A missing task is counted as pending; duplicate completed entries are reported. The merge rejects inconsistent positive and negative decisions for a case.

The field `all_pairs_resolved` means that both the pending and unresolved counts are zero. It alone does not mean all cases have been excluded: the positive count must also be checked. The field `solver_reported_exclusion` records whether the solver has reported a negative decision for every case. It does not assert independent proof verification.

To request an orderly stop, issue this command from another terminal:

```
./build/fan_ramsey17 stop --results results/machine01
```

Allow the running process to finish and return to the shell before merging or copying its checkpoints. The next explicit run clears the stop request. Lock directories such as `RUN.lock` and the individual task locks protect against concurrent writers. If a process was killed, a lock may remain; remove only the stale lock after confirming that its process has terminated. Do not delete checkpoints to recover a stale lock, and do not let two running processes write the same results directory.

8 Distributed computation on six machines

Use exactly the same code and data on every machine. Select `-machines 6` everywhere and a distinct `-machine` number from 1 through 6. Task t , numbered from zero, belongs to machine $(t \bmod 6) + 1$. Each machine writes to its own directory. For machine 1, run:

```
./build/fan_ramsey17 run --data data \
--machine 1 --machines 6 --threads 4 \
--symmetry on --seconds 5 --results results/machine01
```

On machines 2 through 6, change both the machine number and the final directory name accordingly, for example `-machine 2` and `results/machine02`. The six workers need not start together. The number of machines is independent of the number of threads per machine. For scheduled cluster jobs, request the appropriate CPU allocation and run the corresponding command inside the batch job. Scheduler directives depend on the local cluster and are not hard-coded into the distribution.

Retry on each machine using its own existing directory and machine number, with `-retry-unknown` and a larger time limit. Once all processes have stopped, collect the six directories on one machine, preserving their distinct names, and merge:

```
./build/fan_ramsey17 merge --data data \
--inputs results/machine01 results/machine02 \
        results/machine03 results/machine04 \
        results/machine05 results/machine06 \
--output results/merged_six_01
```

If a shared filesystem is used, the same six distinct directories can be merged in place after the workers stop. Do not merge only the individual summary files: case-level checkpoints are required.

To change the number of machines, first stop all writers and merge their checkpoints. Work may then be redistributed by the new task-modulo rule. Each new worker needs the existing checkpoint files for its assigned tasks in its own results directory. Preserve the original merged copy until the redistributed run has been checked. Reassigning tasks changes the allocation of work, not the case indices or mathematical search.

9 Combining a full run with the local certificates

After creating a complete merged checkpoint directory, run:

```
python3 combine_results.py --results results/merged_01 \
--output results/combined.json
```

Use `results/merged_six_01` instead for the six-machine example. This program checks the checkpoint dataset, task ranges, status integrity and case-level coverage, and reruns the forty-case verifier.

It requires zero pending cases and zero positive cases. Every unresolved case in the merged run must belong to the fixed set excluded by the local certificates. An unresolved case outside that set causes failure and requires further computation.

For the reported fourth-pass counts, the expected combination is

$$88\,294\,634 \text{ solver exclusions} + 34 \text{ counting exclusions} + 6 \text{ certificate exclusions} = 88\,294\,674.$$

The combined report records the actual split in the reader's run; it need not reproduce the same intermediate unresolved counts. It does not replace old unresolved status values with solver exclusions and does not label the whole computation as independently certified.

The previously reported cumulative solver counts are listed below for comparison. They are not checkpoint data and cannot substitute for the case-level coverage check.

Pass	Excluded by the solver	Unresolved
First	88 294 434	240
Second	88 294 572	102
Third	88 294 620	54
Fourth	88 294 634	40

Each of these summaries reported zero pending cases and zero positive cases. Different hardware and timing can change the intermediate unresolved set without changing the underlying instances.

10 Independent checks and witness verification

The independent Python data validator uses the standard library:

```
python3 tools/search17_validate.py self-test
python3 tools/search17_validate.py data --data data \
--profile-samples 4 --case-samples 12 \
--output results/data_check.json
```

It checks every seed record and the basic catalogue fields, with full profile and indexing checks on the selected samples. The sample counts can be increased. This command is not a second exhaustive canonical generation and does not check all negative solver decisions.

If a run reports a positive case, preserve the stored witness and verify it before drawing a conclusion:

```
python3 tools/search17_validate.py verify \
--input results/machine01/SAT_FOUND.json --order 17 \
--output results/witness_check.json
```

The program directly checks the proposed colouring for forbidden fans. A valid admissible colouring on seventeen vertices would contradict the proposed exclusion, so a positive result must never be discarded because it differs from the reported computation.

For a single seed pair, such as one of the six certified cases, use:

```
./build/fan_ramsey17 pair --data data \
--left-index 3477 --right-index 1235 \
--seconds 60 --symmetry off --output results/pair.json
```

The pair command selects indices in the seed files, not a task number. Its time-limited result is separate from replay of the supplied proof certificate for that pair.

11 Regenerating the seed libraries and catalogue

Carry out regeneration in a working copy. Use an isolated Python environment, for example on Linux:

```
python3 -m venv .venv
. .venv/bin/activate
python -m pip install -r tools/requirements.txt
mkdir -p regenerated
```

The requirements fix igraph 1.0.0, NumPy 2.3.5, texttable 1.7.0 and NetworkX 3.7. Do not use Python's optimisation flag `-O`: some preparation consistency checks are assertions.

11.1 Canonical generation through order nine

```
python tools/canonical_fans.py --validate
python tools/canonical_fans.py --max-order 9 --target 17 --audit \
    --export regenerated/fan9_target17.graph6
```

The first command tests the unrestricted generator against known small graph counts. The second runs the restricted canonical augmentation used in this search. No finite time limit is imposed here; the program refuses to export an incomplete level as a full seed library. The numbers of retained isomorphism classes at orders 0 through 9 are expected to be

1, 1, 2, 4, 11, 34, 155, 941, 8812, 115174.

The restrictions are hereditary fan and clique exclusions. A minimum red degree of seven is imposed only on a completed seventeen-vertex colouring, not on a seed during augmentation.

11.2 Profiles, filters and stable case indexing

To regenerate the eight-vertex library and construct the profile and pair catalogue using the newly generated nine-vertex library, run:

```
python tools/search17_prepare.py --data regenerated/data \
    --source-nine regenerated/fan9_target17.graph6 \
    --canonical-source tools/canonical_fans.py
```

The source-nine option is checked against the fixed nine-vertex seed identity. As a less expensive separate test of profile preparation, one may instead supply `data/fan9_target17.graph6`; this reuses the archived nine-vertex library and does not regenerate that library.

Check the regenerated fixed inputs and the bindings of the forty exceptional cases:

```
python tools/check_data.py --data regenerated/data \
    --cases certificates/cases.json
python tools/search17_validate.py data --data regenerated/data \
    --profile-samples 6 --case-samples 24 \
    --output regenerated/data_check.json
./build/fan_ramsey17 catalog --data regenerated/data
```

The expected cumulative numbers of ordered pairs are:

Stage	Ordered pairs retained
All eight-by-nine seed pairs	1 014 913 288
Maximum-degree compatibility	682 271 903
Cross-edge count intervals	682 057 873
Joint centre-type compatibility	165 925 659
Saturated maximum-degree boundary	88 294 674

The preparation writes a new catalogue, identity information and a corresponding C++ identity header in its output tree. It also generates profile caches and run reports for that invocation. These generated caches are not required in the published archive. Check that the regenerated fixed files agree with the archived identities before using existing checkpoints. If the ordering changes, even when the graphs are isomorphic and the counts agree, old case indices no longer identify the same instances. Do not bypass the executable’s identity checks or mix checkpoints from different catalogues.

12 Replaying and regenerating certificates

For the ordinary verification procedure, use Section 4; certificate regeneration is optional. Each finite-domain certificate uses the degree interval $7 \leq d_R(v) \leq 10$ and the necessary one-for-one swap inequalities for a maximum red $8 + 9$ cut. Here $d_R(v)$ is the red degree of vertex v . The checker validates these finite constraints using the fixed seed adjacency data. The article proves why every admissible seventeen-vertex colouring has such a partition.

The six certificate case indices and their seed indices are:

Case index	Eight-vertex seed	Nine-vertex seed	Search nodes
51 996 400	3477	1235	911
64 095 209	4228	1235	917
69 103 064	4877	68943	1
69 257 548	4893	68943	1
71 867 158	5479	1235	5668
82 561 944	6899	30887	1

The CMake build in Section 5 also builds the certificate generator `build/fan_tail_certify`. Regenerate all local proof data in a new directory with:

```
python3 tools/regenerate_certificates.py \
  --binary build/fan_tail_certify \
  --output generated_certificates --seconds 120
```

For a Visual Studio build, use `build/Release/fan_tail_certify.exe` as the binary path. The helper rebuilds the thirty-four-case counting certificate, selects the six remaining cases from the fixed input list, calls the C++ generator without symmetry restrictions, and independently replays all regenerated certificates. The output directory must be empty. The time limit is per case; if a certificate is not produced before the limit, use a larger limit and a new output directory.

The regenerated files can also be checked in a separate invocation:

```
python3 verify40.py \  
  --structural generated_certificates/theory_34_certificate.json \  
  --csp-dir generated_certificates \  
  --report results/regenerated_verified40.json
```

Certificate generation must disable the optional twin-row symmetry restriction, because the replay checker verifies a complete branching tree without invoking a symmetry argument. A timeout or incomplete tree is not an exclusion certificate. Store newly generated certificates in a fresh directory and replay them before using them. The helper enforces this symmetry setting. The archived certificates are sufficient to reproduce all six independent exclusions without rebuilding the generator.

13 Submission and online archive

The article and this supplement are intended to accompany the journal submission. The separate reproducibility archive contains the final source code, fixed inputs, independent checkers and certificate data; it also includes this guide so that the archive is self-contained. Research notebooks, obsolete executables, intermediate checkpoints, local dependency installations and development logs are not part of that distribution. Required third-party licences are retained.

The planned public repository is Zenodo, <https://zenodo.org/>. The record has not yet been deposited and no public record DOI is asserted here. Zenodo permits a DOI to be reserved in a draft upload; the authors can then insert that persistent record link in the article and in this supplement before releasing the files. The DOI of the supplementary archive is distinct from the DOI eventually assigned to the article. See the official instructions at <https://help.zenodo.org/docs/deposit/describe-records/reserve-doi/>.